Decisions, repeats and functions: if, for, and a tested function

How do we make Python decide and repeat, so we never copy a formula down thousands of rows?

AI and Data Science Professional | Session 6 of 24 | Module B: Python for data

KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Write a conditional statement with if, elif and else, using Boolean expressions with comparison and logical operators

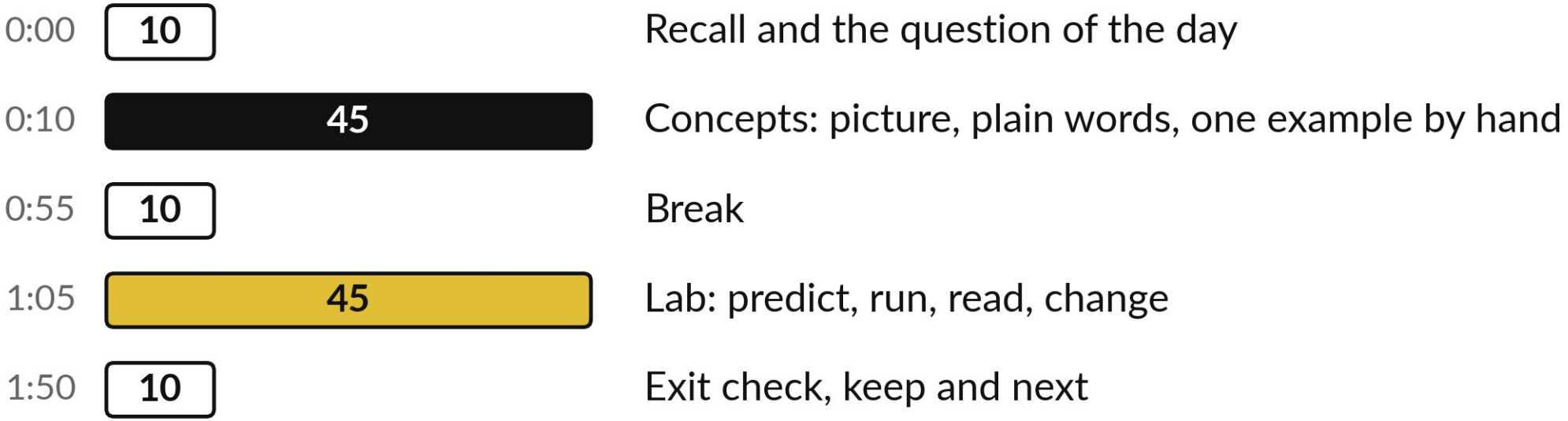


Repeat steps with a for loop and an accumulator, and wrap steps in a function with a parameter and a return value



Test the function with test cases that include the edge cases (boundary values), using assert, and trace code by hand

Your two hours: concepts first, then a Python notebook lab



Pictures and plain words first. Then you open the notebook. Then the tests and what you keep.

Three questions from last time

1. An error ends with: NameError: name 'price' is not defined. Which line do you read first, and what does it tell you?

2. What does '3200' + '8400' give in Python, and why?

3. A list and a dictionary both hold several values. How do you find one value in each?



Answer aloud first. Then we check together.

Answers to the three questions from Session 5

Question	Short answer
An error ends with: NameError: name 'price' is not defined. Which line do you read first, and what does it tell you?	The last line. Python has nothing stored under the name price: check the spelling, or run the cell that creates it.
What does '3200' + '8400' give in Python, and why?	'32008400'. Both values are strings, so plus joins them. Convert them with int() first to add.
A list and a dictionary both hold several values. How do you find one value in each?	In a list by its index, counting from 0. In a dictionary by its key.

Words we use today

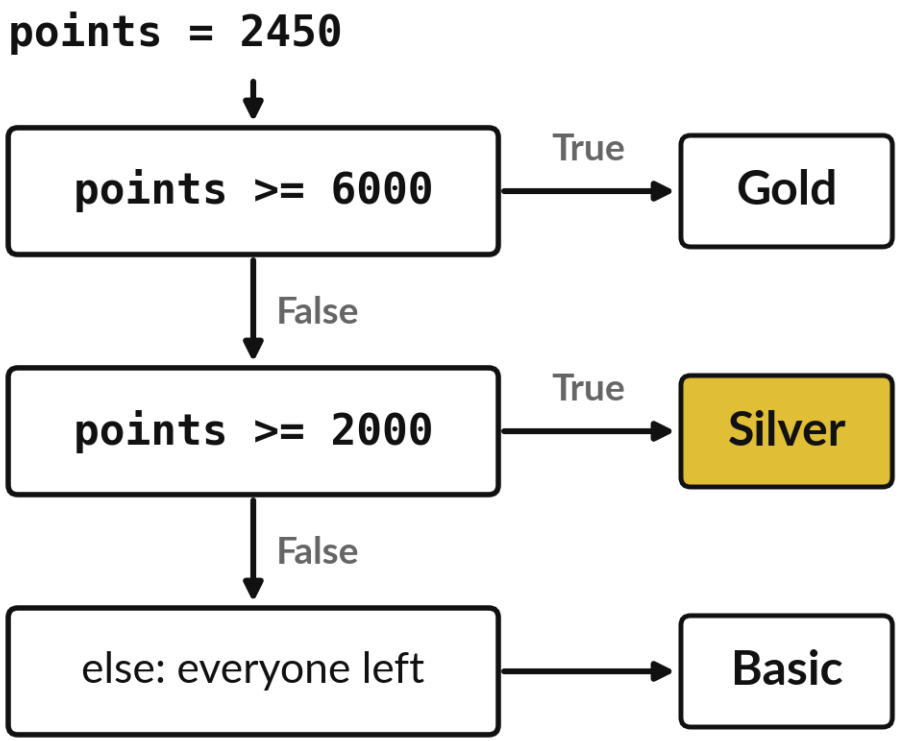
	Word	What it means
	**Conditional statement (if, elif, else)**	Code that runs one branch, chosen by a test that is True or False. It is the IF function written as code.
	**Boolean expression**	A test that gives True or False, built with comparison operators such as >= and logical operators such as and.
	**Iteration (for loop)**	Repeating the same steps for each item in a list. It is fill down written as code.
	**Function (parameter, argument, return value)**	A named set of steps. The parameter is the name for the value that comes in, the argument is the value you pass, and the return value is the answer.
	**Edge case (boundary value) and test case**	An edge case sits exactly where a rule changes. A test case pairs an input with the output you expect.

The rule we will code: the Lakeside Mart loyalty tiers

Tier	Points in 12 months	UGX spent in 12 months
Basic	0 to 1,999	Under 2,000,000
Silver	2,000 to 5,999	2,000,000 to 5,999,999
Gold	6,000 or more	6,000,000 or more

1 point for every UGX 1,000 spent. Source: loyalty_programme.txt, one of the Lakeside Mart policy files.

A conditional statement chooses one branch: Python takes the first test that is True




```
if points >= 6000:
    tier = "Gold"
elif points >= 2000:
    tier = "Silver"
else:
    tier = "Basic"
```

Conditional statement: if, elif and else.

Tests run from the top. The first True branch runs.

elif means else if. else catches the rest.

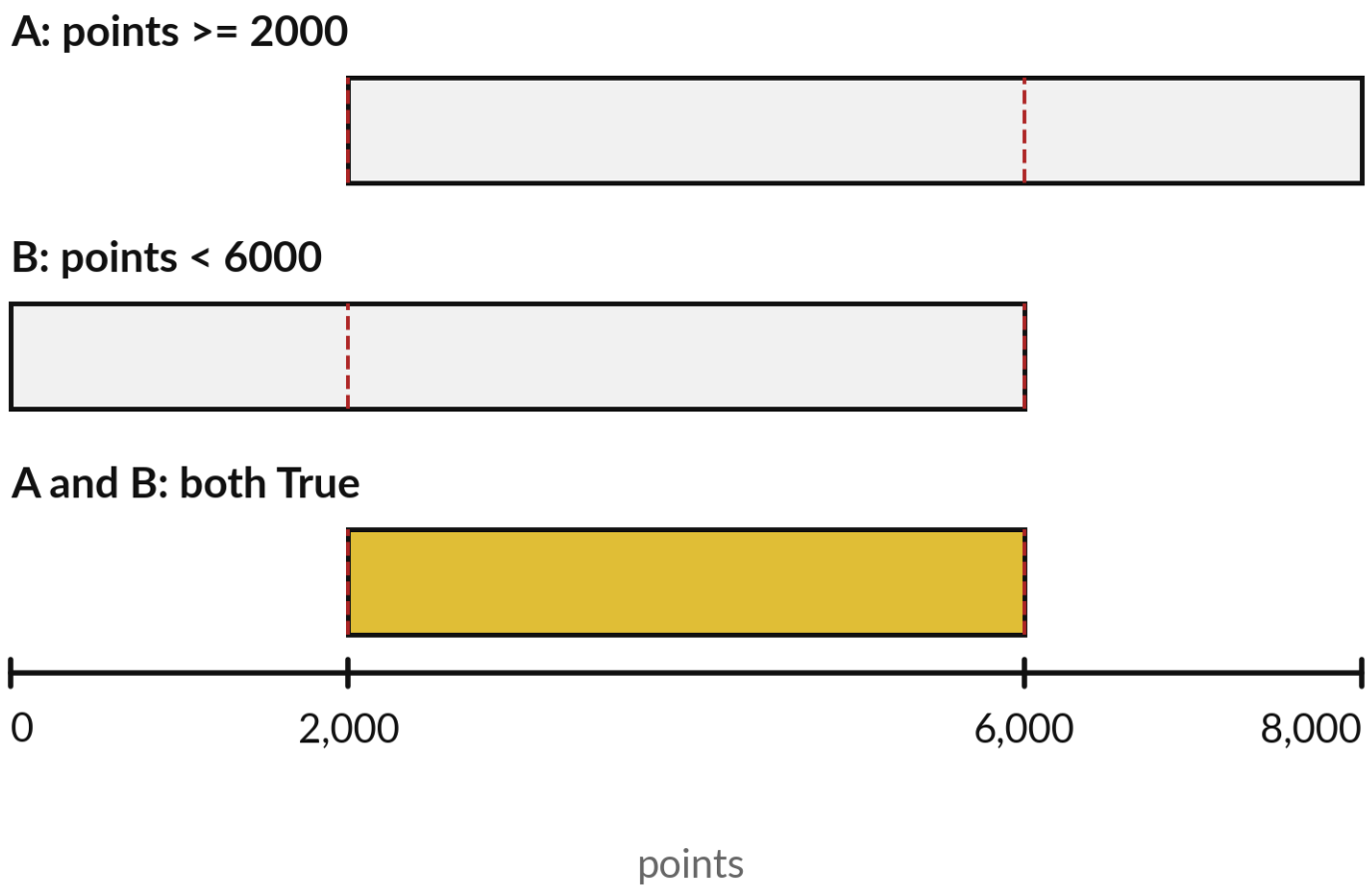
The indent shows which lines belong to which test.

A Boolean expression is a test that gives True or False, built with comparison operators

	Python	Excel	Meaning	With points = 2450
	**points == 2450**	=A2=2450	Equal to	True
	**points != 2000**	=A2<>2000	Not equal to	True
	**points < 6000**	=A2<6000	Less than	True
	**points <= 2450**	=A2<=2450	Less than or equal to	True
	**points > 2450**	=A2>2450	Greater than	False
	**points >= 2000**	=A2>=2000	Greater than or equal to	True

One equals sign stores a value. The comparison equal to is two equals signs. Excel uses one for both jobs.

Logical operators and, or and not combine tests, like AND and OR in Excel



Logical operators: and, or, not.

and is True only when both tests are True.

2,450 points: A and B is True.
6,000 points: it is False.

or needs one True. not flips True and False.

Read the recipe: if, elif and else decide a tier

```
points = 2450
if points >= 6000:
    tier = "Gold"
elif points >= 2000:
    tier = "Silver"
else:
    tier = "Basic"
print(points, "points is", tier)
```

```
2450 points is Silver
```

In plain words

The first test is False, so Python skips Gold.

The second test is True, so tier becomes Silver.

Python skips the else.

The indent and the colon mark each branch.

print runs once, whichever branch ran.

A for loop repeats the same steps for each item, like fill down in a sheet

Spreadsheet: fill down

spend_ugx	points
46,660	46
2,388,850	2,388
82,700	82
2,757,805	2,757
26,650	26

```
=INT(A2/1000)
```

Python: for loop

```
for spend in spends:
    points = spend // 1000
```

The indented lines run once for each item in the list. Five items or five thousand: the code stays the same.

Iteration: repeating steps for each item.

The for line takes one item at a time.

The indented lines run once per item.

// keeps the whole part: one point per UGX 1,000.

Read the recipe: a loop over five customer spends

```
spends = [46660, 2388850, 82700, 2757805, 26650]
for spend in spends:
    points = spend // 1000
    print(spend, "UGX is", points, "points")
```

```
46660 UGX is 46 points
2388850 UGX is 2388 points
82700 UGX is 82 points
2757805 UGX is 2757 points
26650 UGX is 26 points
```

In plain words

spends is a list of five 2025 totals from orders.csv.

The loop takes one spend at a time.

The indented lines run once for each item.

print shows the spend and its points.

An accumulator builds a running total inside a loop, like SUM or a running balance

The code, with line numbers

```
1  orders = [2289600, 63300, 35950]
2  total = 0
3  for value in orders:
4      total = total + value
5  print(total)
```

What happens, one step at a time

Pass	value	total after the pass
start		0
1	2,289,600	2,289,600
2	63,300	2,352,900
3	35,950	2,388,850

Customer C1322 placed three orders in 2025. The accumulator ends at 2,388,850 UGX.

A function is a named set of steps: a parameter goes in and a return value comes out

```
def loyalty_points(spend_ugx):
    """UGX spent to points."""
    points = spend_ugx // 1000
    return points
```

1. def names the function. spend_ugx is the parameter.
2. Docstring: one line saying what it does.
3. The body: indented lines that run when you call it.
4. return sends the answer back.

Call the function

```
loyalty_points(2388850)
```

→ **2388**

return value

The value in the brackets is the argument.

Function: a formula you name yourself.

Parameter: the name for the value that comes in.

Argument: the value you pass when you call it.

return sends the value back to the caller.

Read the recipe: the loyalty_tier function

```
def loyalty_tier(spend_ugx):
    """Return Basic, Silver or Gold."""
    points = spend_ugx // 1000
    if points >= 6000:
        return "Gold"
    elif points >= 2000:
        return "Silver"
    else:
        return "Basic"

print(loyalty_tier(2388850))
print(loyalty_tier(46660))
```

```
Silver
Basic
```

In plain words

def, the name and the parameter spend_ugx.

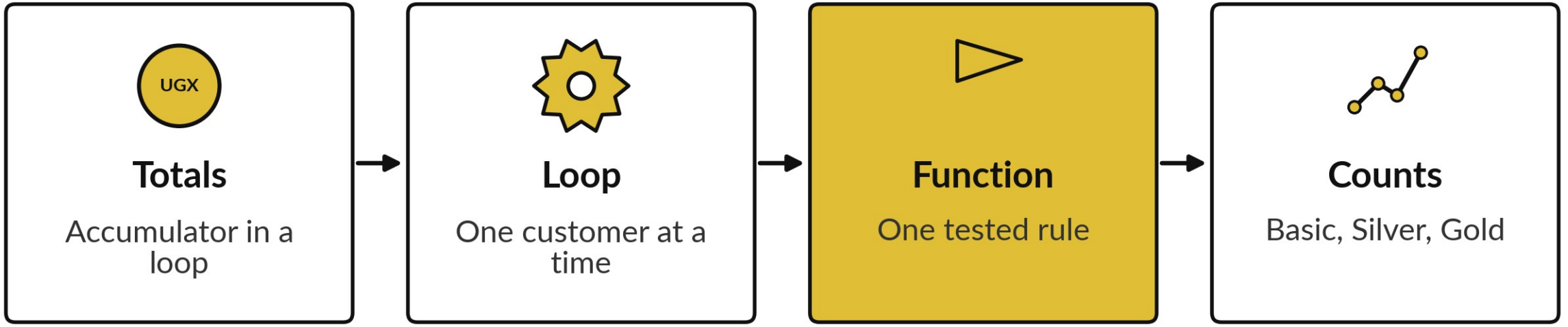
The docstring says what it does.

The body works out points, then tests Gold first.

return gives the tier back.

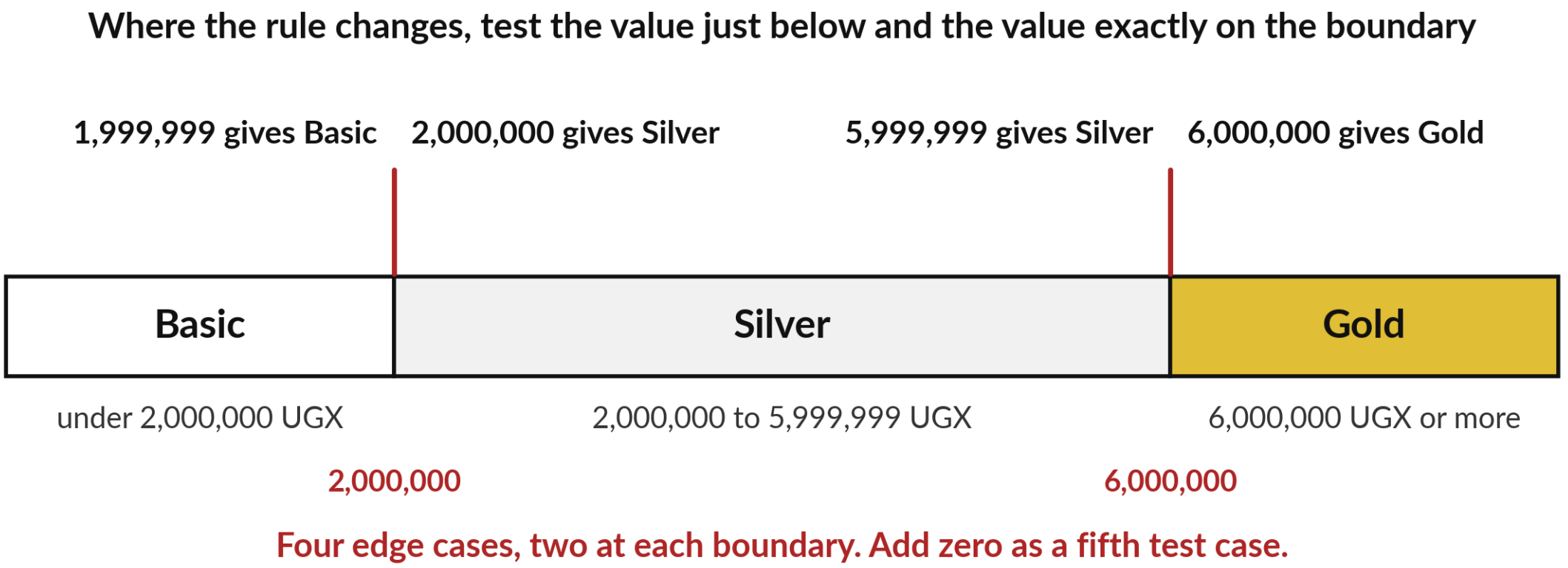
Two calls: 2,388,850 gives Silver, 46,660 gives Basic.

Write the function once, then call it for every customer



A loop repeats the work. A function holds the rule. Together they answer the question for six customers or six million.

An edge case (boundary value) sits exactly where the rule changes



Off-by-one error: writing > where the rule needs >=. Only the boundary value exposes it.

A test case pairs an input with the expected output, and it passes or fails

Test case	spend_ugx	Expected	Got	Result
Zero spend	0	Basic	Basic	PASS
One shilling below Silver	1,999,999	Basic	Basic	PASS
Exactly Silver	2,000,000	Silver	Silver	PASS
One shilling below Gold	5,999,999	Silver	Silver	PASS
Exactly Gold	6,000,000	Gold	Gold	PASS

Expected comes from the policy. Got is what the function returned.

Read the recipe: assert stops with an AssertionError when a test fails

```
assert loyalty_tier(0) == "Basic"
assert loyalty_tier(1999999) == "Basic"
assert loyalty_tier(2000000) == "Silver"
assert loyalty_tier(5999999) == "Silver"
assert loyalty_tier(6000000) == "Gold"
print("All 5 tests pass")
```

```
All 5 tests pass
```

In plain words

assert checks that a test is True.

If it is True, Python carries on silently.

If it is False, Python stops with an AssertionError.

Five asserts, five test cases.

The last line prints only if all five passed.

Trace code by hand to find a fault: tier_v1 tests Silver before Gold

The code, with line numbers

```
1  def tier_v1(spend_ugx):
2      points = spend_ugx // 1000
3      if points >= 2000:
4          return "Silver"
5      elif points >= 6000:
6          return "Gold"
7      else:
8          return "Basic"
```

What happens, one step at a time

Step	Line	What Python does
1	2	points = 7500000 // 1000 stores 7500
2	3	7500 >= 2000 is True
3	4	return "Silver": the function ends
4	none	The Gold test on line 5 is never reached

A colleague's tier_v1 returns Silver for 7,500,000 UGX. The correct tier is Gold.

The same test cases catch the fault: tier_v1 fails 1 of 5

Test case	spend_ugx	Expected	Got	Result
Zero spend	0	Basic	Basic	PASS
One shilling below Silver	1,999,999	Basic	Basic	PASS
Exactly Silver	2,000,000	Silver	Silver	PASS
One shilling below Gold	5,999,999	Silver	Silver	PASS
Exactly Gold	6,000,000	Gold	Silver	FAIL

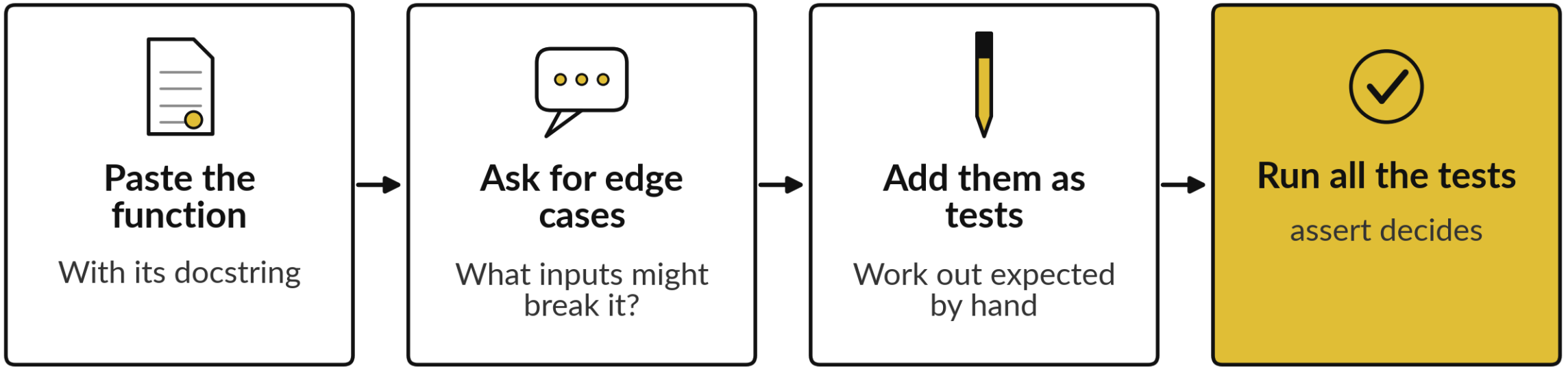
A failed test shows the input, the expected output and the actual output.

Four common mistakes, and the message Python gives for each

What you wrote	What Python says	Fix
if points = 2000:	SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?	Use == to compare.
A function with no return line	print(...) shows None	Add return, so the caller gets the value.
A spend written as text, 2,388,850	TypeError: unsupported operand type(s) for //: 'str' and 'int'	Remove the commas and convert with int().
An if with nothing indented under it	IndentationError: expected an indented block after 'if' statement on line 2	Indent the lines that belong to the if.

Messages copied from real runs. Newer Python versions may word a message slightly differently.

An assistant can review your function, and your own test cases decide whether it is right



An assistant may miss the policy or misread a boundary. Never paste personal or confidential data into a public chat tool.

Where you will use this at work: write the rule once, test it, run it on every record

Your job	The rule	What today lets you do
Credit officer at a bank or microfinance, using Python	Illustrative: a risk band from a score, with cut-offs.	Write the bands as one function and test each cut-off.
Loyalty analyst at a retailer	Tiers from spend. In our 5 percent sample of 2025 orders, 9 of 2,453 customers reach Silver. Sample spend is far below real annual spend.	Apply one tested function to thousands of customers.
Health records officer at a district office	Illustrative: flag a month when visits fall below a threshold.	Loop over the months and test the threshold month.
Officer at a SACCO, using Excel and Python	Illustrative: a fee band by contribution.	Replace a nest of IFs with a function a colleague can test.

Read the recipe before the lab: count customers by tier

```
totals = {'C0009': 46660, 'C1322': 2388850}
counts = {'Basic': 0, 'Silver': 0, 'Gold': 0}
for customer in totals:
    tier = loyalty_tier(totals[customer])
    counts[tier] = counts[tier] + 1
print(counts)
```

```
{'Basic': 1, 'Silver': 1, 'Gold': 0}
```

In plain words

totals is a dictionary: customer id to spend.

counts starts at zero for each tier.

The loop takes one customer id at a time.

It calls loyalty_tier and adds 1 to that tier.

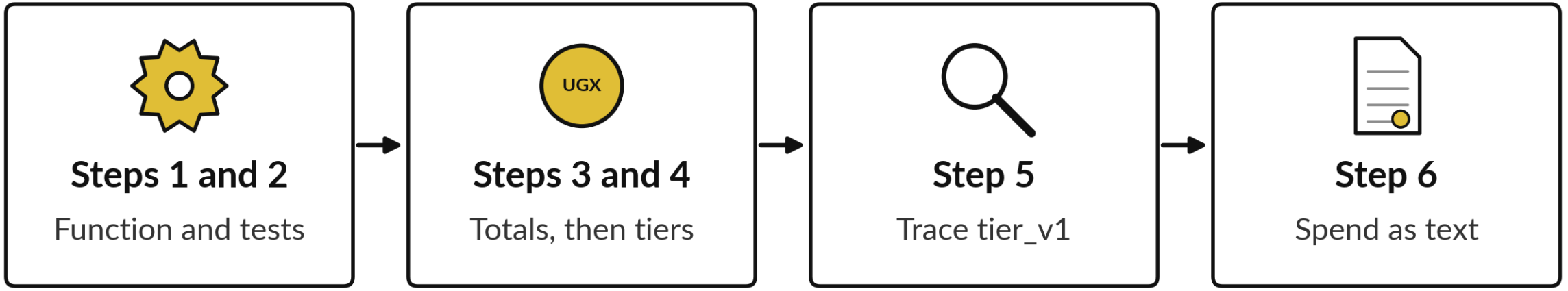
print shows the counts.



Part 2: the Python lab

Close the slides. Open the notebook.

The lab has six steps, and you keep a tested function at the end



For every task: predict, run, read the output, change one thing.

Check your numbers before you move on

	What you compute	You should get
🚩	**Tier for 46,660, for 2,388,850 and for 7,500,000 UGX**	Basic, Silver and Gold
✅	**A and B for 2,450 points, and for exactly 6,000 points**	True and False
🕒	**Test cases that pass for your function**	5 of 5
🟡	**Totals for C0009, C0010, C0021, C0230, C0947, C1322 and all six together**	46,660, 82,700, 26,650, 2,199,645, 2,757,805, 2,388,850; all 7,502,310
🐾	**Customers in Basic, Silver and Gold**	3, 3 and 0
❌	**tier_v1 for 7,500,000, and test cases it fails**	Silver, and 1 of 5
📄	**The spend text 2,388,850 as an int, and its tier**	2,388,850 and Silver

If your number differs, find the task that went wrong. Do not change a number to match.

Five likely mistakes in the lab and how to fix them

What you see	What it means	What to do
IndentationError	Lines under a def, if or for are not indented.	Indent the body by four spaces.
The function prints a tier, but the result is None	The function has no return line.	Use return, not print, inside the function.
A check says NOT YET for the edge cases	The test uses > where the rule needs >=.	Compare with >= for 2,000 and 6,000 points.
AssertionError on a test case	The function returned a different tier than expected.	Read the failing line, trace that one input by hand.
TypeError on //	The spend is a string.	Clean the commas, then use int().

Exit check: three questions, then compare with the short answers

	Question	Short answer
	**What is the tier for exactly 6,000,000 UGX, and why is it a good test case?**	Gold. It is an edge case: exactly on the boundary, where a greater than slip would give Silver.
	**What is the difference between a parameter and an argument?**	The parameter is the name inside the function, such as spend_ugx. The argument is the value you pass, such as 2,388,850.
	**An accumulator holds 0 before the loop. What does it hold after the pass over each of three orders?**	It holds the running total: the first order, then the first two together, then all three.

Save your notebook to Drive, with its tested function

What you keep from today

Keep your saved notebook: the loyalty_tier function with its docstring, five test cases that pass, six customers sorted into tiers, and a fault you traced in a colleague's version.

Our question today

How do we make Python decide and repeat, so we never copy a formula down thousands of rows?

Next session

Session 7: pandas I, load, look and filter. We open a file of 30,229 orders and ask the first questions of it.

Take away

Your lab file

A tested function

Your test cases

Five, with the edge cases

Next session

pandas: load, look, filter